Lab-1: Accessing the Camera via ROS2

Lester, Ching-I

In this lab, we will use ROS2 to activate a depth camera. You are encouraged to use Linux/Ubuntu for further applications, especially in robotics. However, a virtual machine will also be provided for those using Windows. In this tutorial, you will learn how to activate a depth camera and record a rosbag for logging the data.

The tutorial includes two parts:

A. Set up the environment.
   A-1. Linux/Ubuntu dual system
   A-2. WSL2 on Windows
B. Set up the D415 camera and record data.

This document is also uploaded to GitHub. Tutorial/lab1

A-1. Linux/Ubuntu dual system

(recommended but optional, bonus 10 bonus points)

Installation issues may vary by computer and involve deeper system access, which carries some risk but will expand your engineering knowledge. You are encouraged to complete the installation independently.

If you need help, please email our TAs. While they will assist as much as possible, not all issues can be guaranteed to be resolved. Additionally, TAs may adjust grades based on the level of assistance required, as we aim to foster independent problem-solving.

1 Reminder

Please review the important reminder first to ensure that your PC meets the required specifications.

1.1 Please check the windows Bitlocker is disabled (check this link for more information https://www.asus.com/support/faq/1044341/)
1.2 You need to login into your Bios to disable the Fast boot and Security boot (if your security boot is locked you need to set an admin password first).
1.3 Check wifi card: In windows, you can use ctrl + shift + esc to see the task manager, check the wi-fi card name (if you are not intel wifi card you need to share the network with your phone via cable).
1.4 If you meet black screen problem when you try to install, choose the safe graphic when using the USB boot device.
1.5 When installing not to choose any option of Erase!!! It will empty your windows. (irreversible)

2 Install Ubuntu

After reviewing the important reminder, you will need the following:

2.1 Bootable USB drive *1 (minimum 16GB; any Ubuntu version is acceptable, though we recommend version 22.04 for Intel 13th/14th gen CPUs)

2.2 Your computer equipped with an NVIDIA GPU (e.g., GTX or RTX series)
(If your computer does not have a GPU or does not use NVIDIA, please use the A-2 WSL.)

Before proceeding with installation, ensure the following:

1) BitLocker is disabled (if applicable)
2) Compatibility of your Wi-Fi card (refer to the important reminder)

You can follow the video to install dual system
https://www.youtube.com/watch?v=tEh1RfmbTBY
*If you install a dual system, recommend having ⅓ of your storage size (minimum 128GB upon) to ubuntu.

3 Install required utils

3.1 Update the system:

You can use ctrl + alt + t to pop out the terminal $ means type the command to install Terminator, Curl, Git, vim, net-tools

```
$ sudo apt update
$ sudo apt install terminator curl git vim net-tools
```

3.2 Install nvidia graphic drive

You can choose any driver version which is compatible to your computer.(e.g. 525 535 555)

```
$ sudo apt update
$ sudo ubuntu-drivers list
$ sudo apt install nvidia-driver-5xx
$ sudo reboot now
```

A few computer will freeze at logo, if you met this problem please choose ubuntu advanced option > and choose another kernel version (Without safe mode)

You can use $ nvidia-smi to check your graphic card is working correctly

4 Install Docker

https://docs.docker.com/engine/install/ubuntu/

```
$ curl -fsSL https://test.docker.com -o test-docker.sh
$ sudo sh test-docker.sh
```

Fix daemon permission denied problem when you using docker command

```
$ sudo groupadd docker
$ sudo usermod -aG docker $USER
$ sudo reboot now
```

5 Install Nvidia Docker toolkit

https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html

```
$ curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo gpg --dearmor -o
/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg \
  && curl -s -L
https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list | \
    sed 's#deb https://#deb
[signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] https://#g' | \

$ sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
$ sudo apt-get update
$ sudo apt-get install -y nvidia-container-toolkit
```

6 Setup git ssh-key (you need to have a github account first)

https://ithelp.ithome.com.tw/articles/10205988

You can use $ ssh -T git@github.com to check your key is correctly setup

Check https://github.com/hrc-pme/Deep_learning_d415 for B part full tutorial.

A-2. WSL

If you do not have enough storage space for a dual-boot system, you can use WSL 2 (Windows Subsystem for Linux) on Windows. This section will guide you through installing WSL 2, Ubuntu, and Docker Desktop, and configuring them so that the ROS 2 camera environment can run successfully.

1 Install WSL 2 (You can also get more information in WSL_Install)

1.1 Open PowerShell as Administrator

1.2 Run the following command to install WSL and the default Ubuntu distribution:

```
$ wsl --install
```

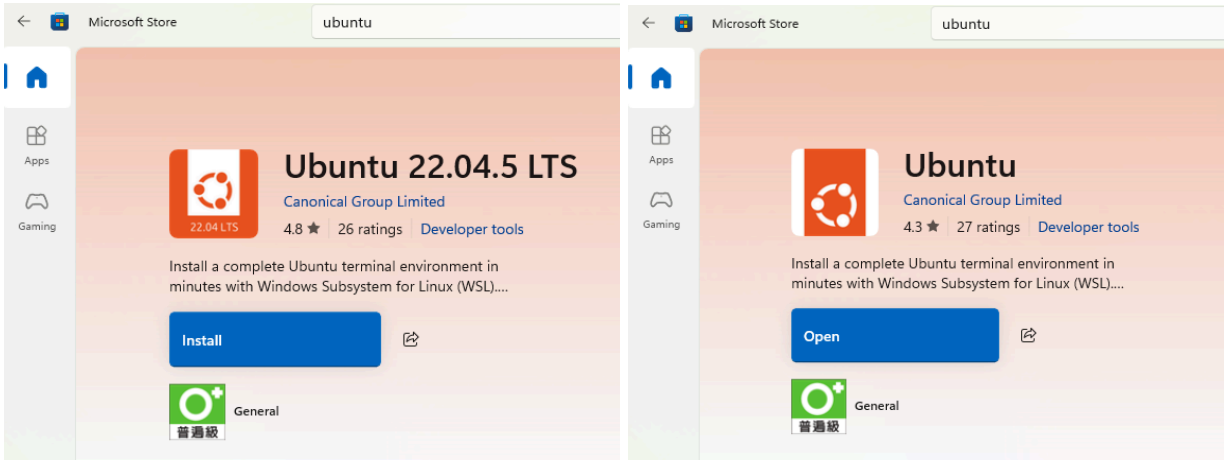
1.3 Verify your WSL version:

```
$ wsl –version                    # Ensure that your version is at least 2.0
```

1.4 Restart your computer when prompted

2 Install Ubuntu for WSL

2.1 Open the Microsoft Store, search for Ubuntu 22.04 LTS, and click Install. (or Ubuntu is for sure)



2.2 After installation, launch Ubuntu:

You can open it from the Start Menu, or In PowerShell, type:

```
$ ubuntu
```

2.3 Set your username and password when prompted

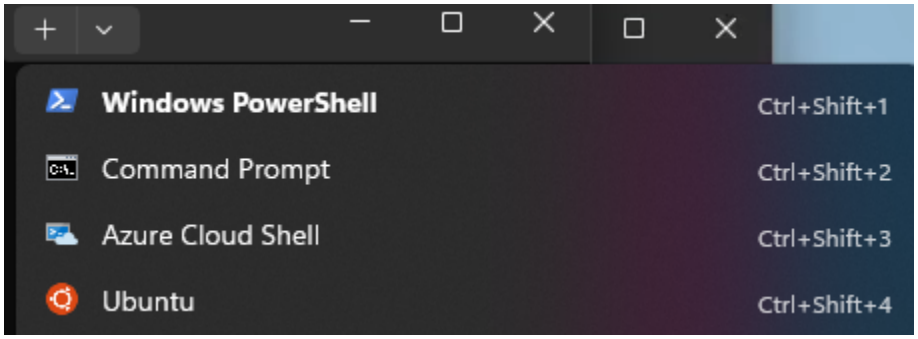
```
Installing, this may take a few minutes...
Please create a default UNIX user account. The username does not need to match your Windows username.
For more information visit: https://aka.ms/wslusers
Enter new UNIX username:
```

```
New password:
Retype new password:
```

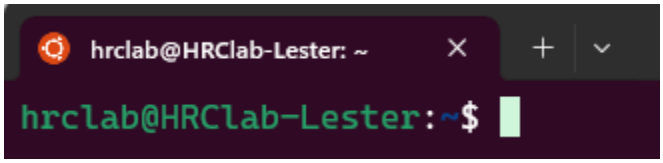
(Notice that password will not be show here)

2.4 Update the system:
`$ sudo apt update && sudo apt upgrade -y`

2.5 The next time you want to open WSL:
Type Ubuntu in the Windows search bar, or
Open PowerShell and type:
`$ ubuntu`



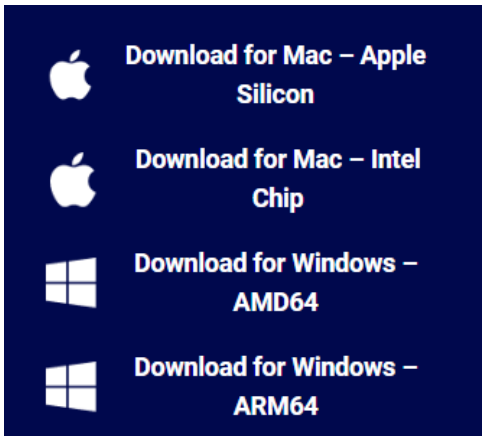
2.6 If successful, your terminal will look like a standard Ubuntu shell, and you can start using Linux commands.



For example, to open Visual Studio Code inside WSL:
`$ code .` # to start VScode

3 Install Docker-Desktop

3.1 Download Docker Desktop for Windows from the official site:
(Docker Desktop)
During installation, most users should select AMD64 (x64).
You can confirm your system type in Device Manager → System Information (If it shows *x64-based processor*, select AMD64.)

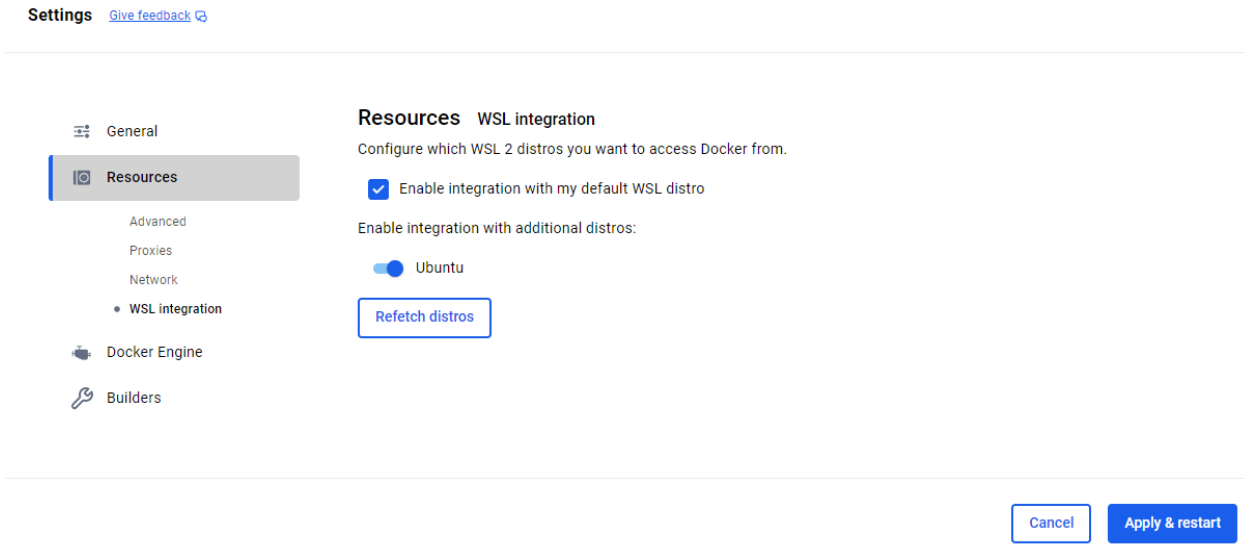


3.2 After installation, open Docker Desktop (Docker Desktop must remain open while you are using it)

3.3 Change the setting

Go to Settings → Resources → WSL Integration, enable "Enable integration with my default WSL distro" and ensure Ubuntu is selected.

Then click Apply & Restart.

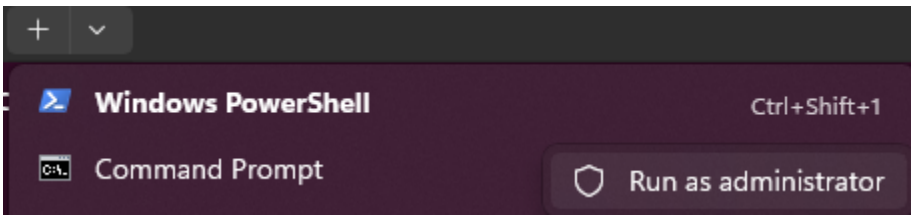


4 Connect the Camera to WSL

This step allows your WSL environment to access a USB camera (e.g., Intel RealSense)

4.1 Open PowerShell as Administrator and run

$ `winget install --interactive --exact dorssel.usbipd-win`



4.2 Find the camera, list all connected USB devices:

$ usbipd list # find the camera BUSID

```
PS C:\Users\HRCla> usbipd list
Connected:
BUSID  VID:PID    DEVICE                                                        STATE
1-11   1462:7d98  USB 輸入裝置                                                  Not shared
1-14   8087:0033  Intel(R) Wireless Bluetooth(R)                                Not shared
1-25   8086:0ad3  Intel(R) RealSense(TM) Depth Camera 415  Depth, Intel(R) ...  Not shared
```

Find your camera's BUSID from the list.

4.3 Bind the camera:

Example:

$ usbipd bind --busid 1-25

# Replace 1-25 with the actual BUSID of your camera
$ usbipd list

```
PS C:\Users\HRCla> usbipd bind --busid 1-25
PS C:\Users\HRCla> usbipd list
Connected:
BUSID  VID:PID    DEVICE                                                        STATE
1-11   1462:7d98  USB 輸入裝置                                                  Not shared
1-14   8087:0033  Intel(R) Wireless Bluetooth(R)                                Not shared
1-25   8086:0ad3  Intel(R) RealSense(TM) Depth Camera 415  Depth, Intel(R) ...  Shared
```

If it works, it will turn to shared.

4.4 Attach the camera to WSL:

$ usbipd attach --wsl --busid 1-25 --auto-attach

4.5 Verify that the camera is recognized inside Ubuntu (wsl terminal)

$ lsusb

```
hrclab@HRClab-Lester:~$ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 002 Device 006: ID 8086:0ad3 Intel Corp. RealSense D415
```

You should see a device entry corresponding to your camera (e.g., Intel Corp. RealSense D415).

B. Setup the D415 and record data

In this section, we will turn on the D415 camera and record a bag file.

1. Open Vscode or terminal (for wsl user)

1.1 Open terminal

$ ubuntu

1.2 Open Vscode

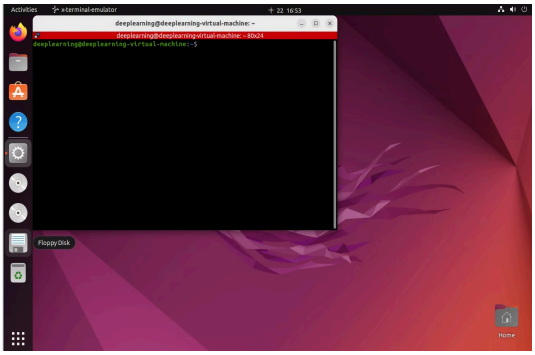
$ code .

1.3 Ctrl + ~ to open the terminal in Vscode

1.4 Remind to open the camera to wsl we just done in A-2-4

2. Open the terminal:

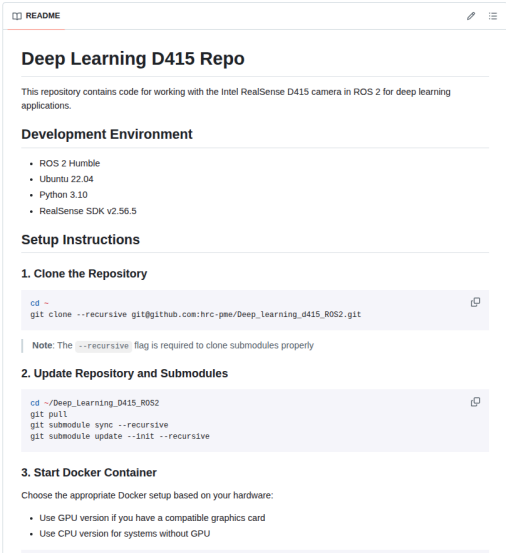
Press Ctrl + Alt + T to open the terminal.



Or using Vscode will be easier Ctrl + ~ to open the terminal in Vscode

3. Set up the camera and record data:

Refer to the "Setup Instructions" section in the README file for detailed setup instructions: https://github.com/hrc-pme/Deep_learning_d415_ROS2



README

Deep Learning D415 Repo

This repository contains code for working with the Intel RealSense D415 camera in ROS 2 for deep learning applications.

Development Environment

- ROS 2 Humble
- Ubuntu 22.04
- Python 3.10
- RealSense SDK v2.56.5

Setup Instructions

1. Clone the Repository

```
cd ~
git clone --recursive git@github.com:hrc-pme/Deep_learning_d415_ROS2.git
```

Note: The `--recursive` flag is required to clone submodules properly

2. Update Repository and Submodules

```
cd ~/Deep_Learning_D415_ROS2
git pull
git submodule sync --recursive
git submodule update --init --recursive
```

3. Start Docker Container

Choose the appropriate Docker setup based on your hardware:

- Use GPU version if you have a compatible graphics card
- Use CPU version for systems without GPU

- **Check points**

1 Record a 10-second image using the rosbag command. Capture a screenshot of the rosbag information as shown below. (rosbag) (80 points)

E.g.

```
root@welly-msi:~/Deep_learning_d415/bags/2024_1023_1918# rosbag info 2024-10-23-19-18-36.bag
path:        2024-10-23-19-18-36.bag
version:     2.0
duration:    10.3s
start:       Oct 23 2024 19:18:36.70 (1729682316.70)
end:         Oct 23 2024 19:18:47.04 (1729682327.04)
size:        19.7 MB
messages:    464
compression: none [25/25 chunks]
types:       sensor_msgs/CameraInfo      [c9a58c1b0b154e0e6da7578cb991d214]
             sensor_msgs/CompressedImage [8f7a12909da2c9d3332d540a0977563f]
topics:      /camera/aligned_depth_to_color/image_raw/compressedDepth   155 msgs    : sensor_msgs/CompressedImage
             /camera/color/camera_info                                  155 msgs    : sensor_msgs/CameraInfo
             /camera/color/image_raw/compressed                         154 msgs    : sensor_msgs/CompressedImage
```

2 ROS message (20 points)

Write a Python or C++ script to subscribe to the color image topic in the rosbag and use rospy.loginfo to display the image timestamp and a notice string.